Lecture 13 - Spooky Dataframes

Let's talk about dataframes

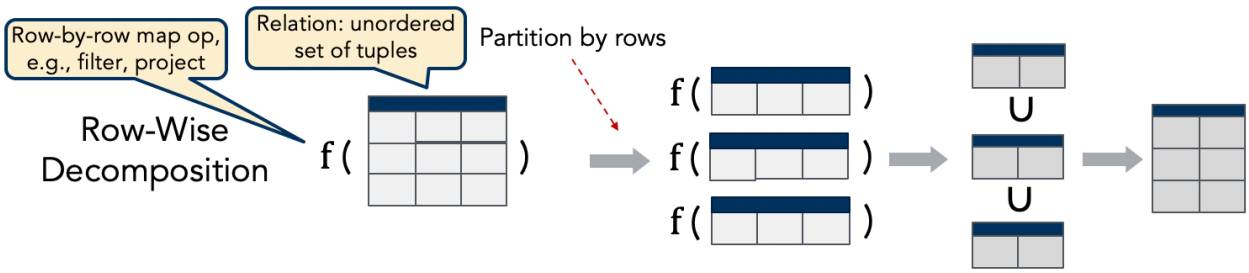
- central to data analysis and data science
  - pandas often cited as the reason for python's popularity
  - called the most important tool in data science
  - used for everything ranging from data cleaning to even primitive ML, and even more especially in combination with ML libraries
- 500+ functions, allowing you to do anything you want to your data
  - Lots of redundancy: many ways to do the same thing (1700x change)
- We identified that pandas was being used to operate on very large datasets and was breaking down
  - Often would OOM
  - Very inefficient w/ memory would, make multiple copies
  - Each operator executed by itself, in entirety, no optimization
  - Eager metadata maintenance
- So we started building Modin, a parallel dataframe system, architected as a "drop-in" replacement for pandas
- Here, the papers came after the system was already gaining traction in the OSS community
- Started a company, which was sold to Snowflake
- 55M+ downloads

Let's talk about the 2020 paper that crystallized the dataframe data model and algebra

- Lots of math in the paper, but at the highest level, a dataframe is a four-tuple (Array, Row Labels, Column Labels, Types)
- Unlike relations:
  - ordered along both rows and columns
  - rows are named
  - types can be unspecified after operations, and may need to be induced (as in spreadsheets)
- From an algebraic standpoint
  - output could have arbitrary schema that depended on data
    - e.g. one-hot encoding or pivot, or dropNA along columns
    - simply a no-no for
  - data was equivalent to metadata, so we could move information from the labels to the data and vice-versa
- We also boiled all the operators down to a small number of operators (we refined this in the second 2021 paper), that included
  - ordered versions of relational operators along both rows and columns
    - e.g., filter along columns
  - to/from labels
  - transpose

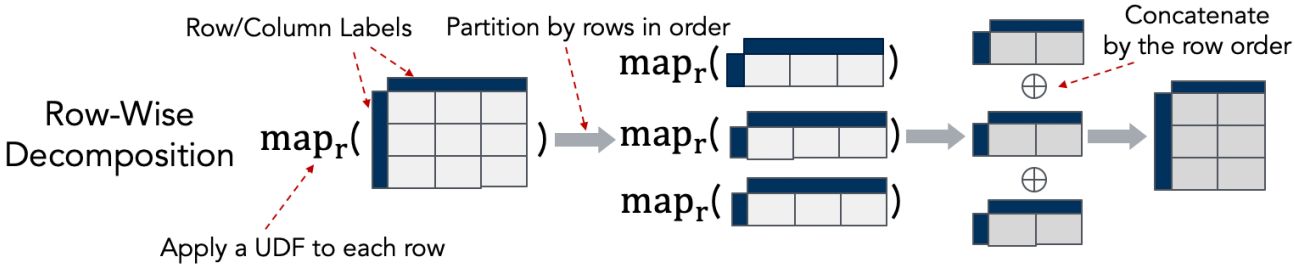
Next, how do we think about parallelizing the evaluation of dataframe operators?

- can we apply the same ideas from parallel databases, e.g., breaking an operation on an entire relation into those on partitions?
- two twists:
  - order and access



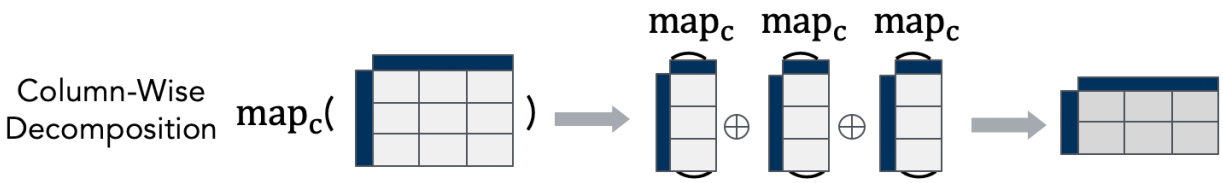
More complicated for blocking/binary ops,
e.g., sort, group by, joins
will focus on the simplest case for this talk

Decomposition Rules for Dataframes: Remember Order!

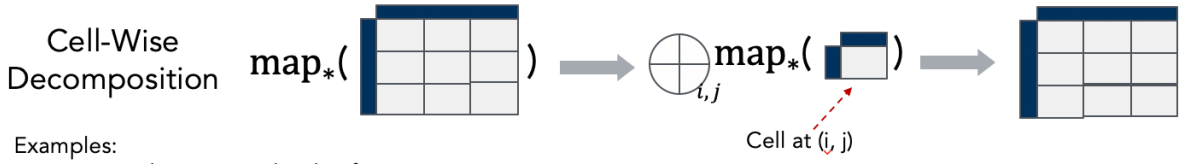


Examples:
- add a derived feature
- explode a row into multiple rows

Decomposition Rules for Dataframes: Remember Access!

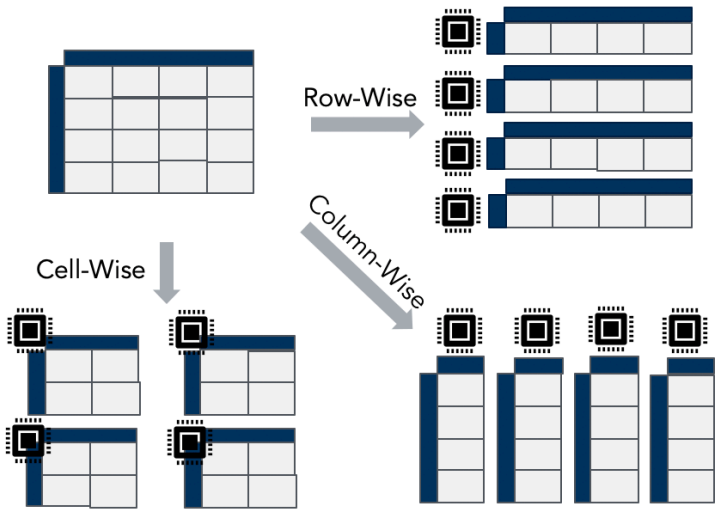


Examples:
- Fill the NaN values of each column based on a UDF
- One hot encoding



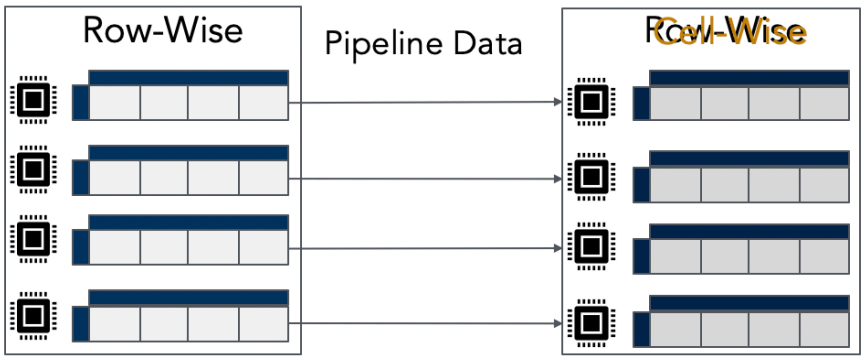
Examples:
- Regex replace across the dataframe

Applying Decomposition Rules for a Single Operator



Applying Decomposition Rules to a Chain of Operators

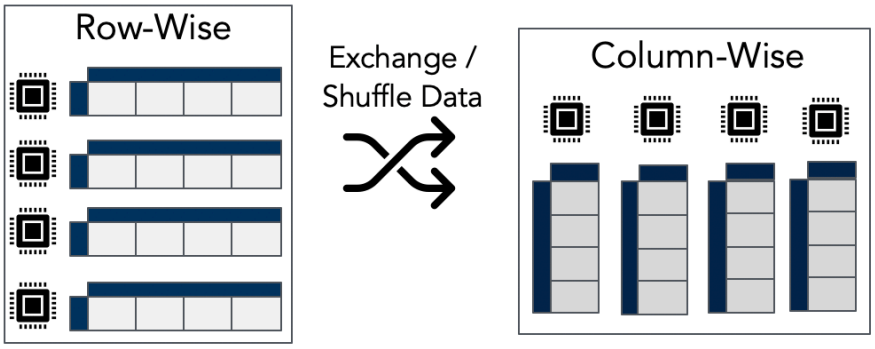
Need to decide communication between successive operators



Cell-wise decomposition is more flexible wrt its input than row/column-wise decomposition

Applying Decomposition Rules to a Chain of Operators

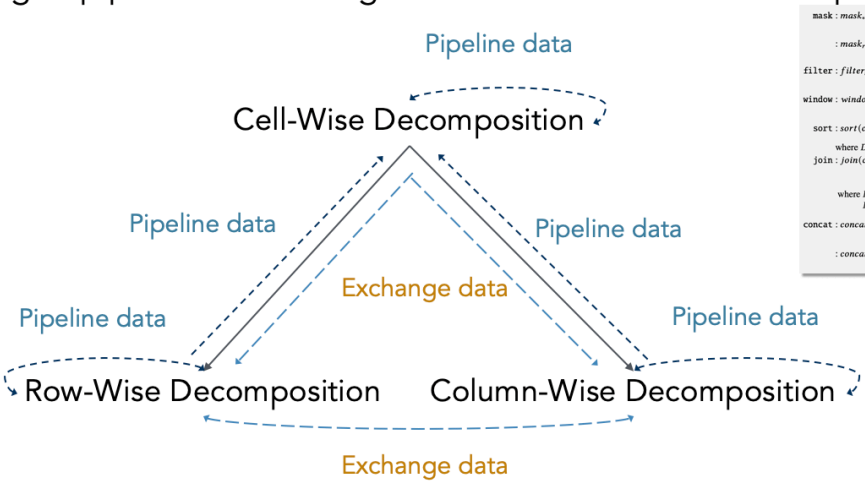
Need to decide communication between successive ops



Exchanging data is much more costly than pipelining

Applying Decomposition Rules to a Chain of Operators

Deciding to pipeline or exchange data between two chained operators

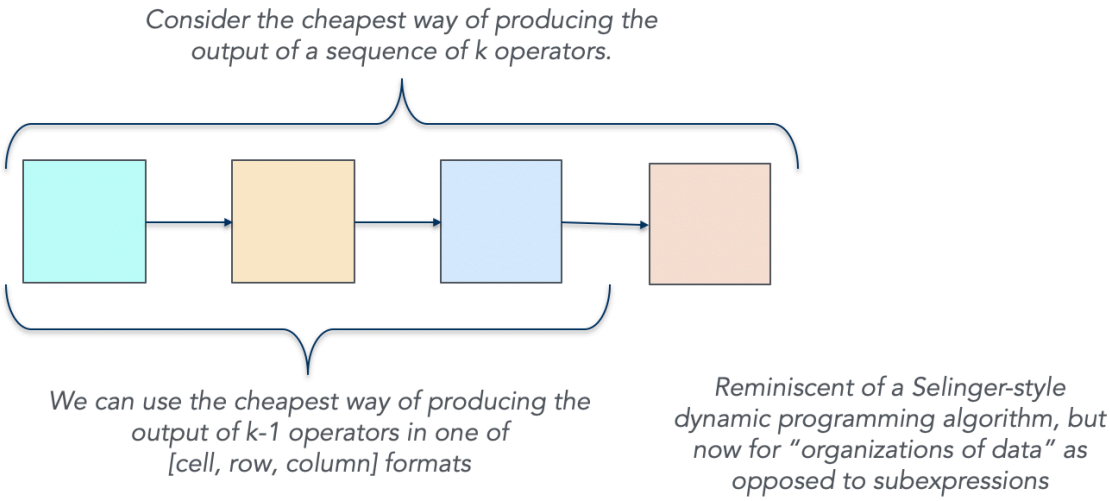


$$
\begin{aligned}
\texttt{mask} &: mask_s(labels, D) = \bigoplus_{i=1}^{m} \bigoplus_{j=1}^{n} h_s^{mask}(labels, D_{ij}) \\
&: mask_r(rnSet, D) = \bigoplus_{i=1}^{m} \mathbb{I}[i \in rnSet] D_{is} \\
\texttt{filter} &: filter_r(f_r^{flt}, D) = \bigoplus_{i=1}^{m} f_r^{flt}(D_{is}) \\
\texttt{window} &: window_r(f_r^{win}, w, D) = \bigoplus_{i=1}^{m} \bigoplus_{j=1}^{n} f_r^{win}(\bigoplus_{k=j}^{j+w} D_{ik}) \\
\texttt{sort} &: sort(cols, D) = \bigoplus_{i=1}^{m} h_n^{sort}(cols, D_{[p_i, p_{i+1})}) \\
&\quad \text{where } D_{[p_i, p_{i+1})} = filter_r(p_i \leq cols < p_{i+1}, D) \\
\texttt{join} &: join(cols^l, D^l, cols^r, D^r) = join(\bigoplus_g D_k^l, \bigoplus_g D_k^r) \\
&\quad = \bigoplus_g cross_prod(D_k^l, D_k^r) \\
&\quad \text{where } D_k^l = filter_r(cols^l = k, D^l) \\
&\quad D_k^r = filter_r(cols^r = k, D^r) \\
\texttt{concat} &: concat_r^{out}(D^1, D^2) = \bigoplus_{k \in \{1,2\}} \bigoplus_{i=1}^{m_k} h_r^{out}(labels_{out}, D_{ri}^k) \\
&: concat_r^{in}(D^1, D^2) = \bigoplus_{k \in \{1,2\}} \bigoplus_{i=1}^{m_k} mask_r(labels_{in}, D_{ri}^k)
\end{aligned}
$$

[actually more complex hierarchy, see paper!]

For a chain of ops how do we communicate in-between to minimize total latency?

Overall Optimization: Simple Optimal Substructure Intuition



Paper also discusses ways to lazily maintain metadata

Vision paper outlines a bunch of open, unaddressed questions - still yet to be done